Занятие 47.2. Цикл foreach

На этом уроке мы рассмотрим использование цикла foreach в языке C++.

Оглавление:

1. Цикл foreach
2. Цикл foreach и ключевое слово auto
3. Цикл foreach и ссылки
4. Еще один пример
5. Цикл foreach и не массивы
6. Цикл foreach не работает с указателями на массив
7. Могу ли я получить индекс текущего элемента?
8. Заключение
9. Тест

Цикл foreach

Мы рассматривали примеры использования **цикла for** для осуществления итерации по каждому элементу массива. Например:

```
#include <iostream>

int main()
{
    const int numStudents = 7;
    int scores[numStudents] = { 45, 87, 55, 68, 80, 90, 58 };
    int maxScore = 0; // сохраняем наивысший балл
    for (int student = 0; student < numStudents; ++student)
        if (scores[student] > maxScore)
            maxScore = scores[student];

    std::cout << "The best score was " << maxScore << "\n";

    return 0;
}
```

В то время как цикл for предоставляют удобный и гибкий способ итерации по массиву, в них так же легко можно запутаться и наделать кошмаров неучтенных единиц.

Поэтому в C++11 добавили новый тип цикла — **foreach** (или *«цикл, основанный на диапазоне»*), который предоставляет более простой и безопасный способ итерации по массиву (или по любой другой структуре типа списка).

Синтаксис цикла foreach следующий:

```
for (объявление_элемента : массив)
    стейтмент;
```

Выполняется итерация по каждому элементу массива, присваивая значение текущего элемента массива переменной, объявленной как **элемент** (объявление_элемента). В целях улучшения производительности объявляемый элемент должен быть того же типа, что и элементы массива, иначе произойдет **неявное преобразование**. Рассмотрим простой пример использования цикла foreach для вывода всех элементов массива `math`:

```
#include <iostream>

int main()
{
    int math[] = { 0, 1, 4, 5, 7, 8, 10, 12, 15, 17, 30, 41};
    for (int number : math) // итерация по массиву math
        std::cout << number << ' '; // получаем доступ к элементу массива в этой итерации через переменную number

    return 0;
}
```

Результат выполнения программы:

```
0 1 4 5 7 8 10 12 15 17 30 41
```

Рассмотрим детально, как это всё работает. При выполнении цикла foreach переменной `number` присваивается значение первого элемента (т.е. значение `0`). Дальше программа выполняет стейтмент вывода значения переменной `number`, т.е. нуля. Затем цикл выполняется снова, а значением переменной `number` уже является `1` (второй элемент массива). Вывод значения `number` выполняется снова. Цикл продолжает свое выполнение до тех пор, пока в массиве не останется непройденных элементов. В конце выполнения программа возвращает `0` обратно в операционную систему с помощью оператора return.

Обратите внимание, переменная `number` не является индексом массива. Ей просто присваивается значение элемента массива в текущей итерации цикла.

Цикл foreach и ключевое слово auto

Поскольку объявляемый элемент цикла foreach должен быть того же типа, что и элементы массива, то это идеальный случай для использования **ключевого слова auto**, когда мы позволяем C++ вычислить тип данных элементов массива вместо нас. Например:

```
#include <iostream>

int main()
{
    int math[] = { 0, 1, 4, 5, 7, 8, 10, 12, 15, 17, 30, 41 };
    for (auto number : math) // из number определяется автоматически исходя из типа элементов массива math
        std::cout << number << ' ';

    return 0;
}
```

Цикл foreach и ссылки

В примерах, приведенных выше, объявляемый элемент всегда является переменной:

```
    int array[7] = { 10, 8, 6, 5, 4, 3, 1 };
    for (auto element : array) // element будет полной текущего элемента массива
        std::cout << element << ' ';
```

То есть каждый обработанный элемент массива копируется в переменную `element`. А это копирование может оказаться затратным, в большинстве случаев мы можем просто сослаться на исходный элемент с помощью **ссылки**:

```
    int array[7] = { 10, 8, 6, 5, 4, 3, 1 };
    for (auto &element : array) // знак амперсанда делает element ссылкой на текущий элемент массива, предотвращая копирование
        std::cout << element << ' ';
```

В примере, приведенном выше, в качестве объявляемого элемента цикла foreach используется ссылка на текущий элемент массива, при этом копирования этого элемента не происходит. Но, с указанием обычной ссылки, любые изменения элемента будут влиять на сам массив, что не всегда может быть желательно.

Конечно же, хорошей идеей будет сделать объявляемый элемент константным, тогда вы сможете его использовать в режиме *«только для чтения»*:

```
    int array[7] = { 10, 8, 6, 5, 4, 3, 1 };
    for (const auto &element : array) // element - это константная ссылка на текущий элемент массива в итерации
        std::cout << element << ' ';
```

Правило: Используйте обычные ссылки или константные ссылки в качестве объявляемого элемента в цикле foreach (в целях улучшения производительности).

Еще один пример

Вот пример первой программы из начала этого урока, но уже с использованием цикла foreach:

```
#include <iostream>

int main()
{
    const int numStudents = 7;
    int scores[numStudents] = { 45, 87, 55, 68, 80, 90, 58};
    int maxScore = 0; // сохраняем наивысший индекс наибольшего score (оценкиии)
    for (const auto &score : scores) // итерация по массиву, присваивая каждое значение массива поочередно переменной score
        if (score > maxScore)
            maxScore = score;

    std::cout << "The best score was " << maxScore << "\n";

    return 0;
}
```

Обратите внимание, здесь нам уже не нужно вручную прописывать индексацию массива. Мы можем получить доступ к каждому элементу массива непосредственно через переменную `score`.

Цикл foreach и не массивы

Циклы foreach работают не только с **фиксированными массивами**, но также и со многими другими структурами типа списка (хаками, как вектором (например, std::vector), связанные списки, деревья. Не беспокойтесь, если вы не знаете, что это такое (мы все это рассмотрим чуть позже). Просто помните, что циклы foreach обеспечивают гибкий и удобный способ итерации не только по массивам:

```
#include <vector>
#include <iostream>

int main()
{
    std::vector<int> math = { 0, 1, 4, 5, 7, 8, 10, 12, 15, 17, 30, 41}; // обратите внимание здесь на использование std::vect
    for (const auto &number : math)
        std::cout << number << ' ';

    return 0;
}
```

Цикл foreach не работает с указателями на массив

Для итерации по массиву, цикл foreach должен знать длину массива. Поскольку массивы, которые **распадаются в указатель**, не знают своей длины, то циклы foreach с ними работать не могут!

```
#include <iostream>

int sumArray(int array[]) // array - это указатель
{
    int sum = 0;
    for (const auto &number : array) // ошибка компилятора, размер массива неизвестен
        sum += number;

    return sum;
}

int main()
{
    int array[7] = { 10, 8, 6, 5, 4, 3, 1 };
    std::cout << sumArray(array); // array распадается в указатель здесь
    return 0;
}
```

По этой же причине циклы foreach не работают с **динамическими массивами**.

Могу ли я получить индекс текущего элемента?

Циклы foreach не предоставляют прямой способ получения индекса текущего элемента массива. Это связано с тем, что большинство структур, с которыми могут использоваться циклы foreach (например, связанные списки), например не индексируются!

Заключение

Циклы foreach обеспечивают лучший синтаксис для итерации по массиву, когда нам нужно получить доступ ко всем элементам массива в последовательном виде. Эти циклы предпочтительнее использовать вместо стандартных циклов for в случаях, когда они могут использоваться. Для предотвращения создания копий каждого элемента в качестве объявляемого элемента следует использовать ссылку.

Тест

Это должно быть легко!

Объявите фиксированный массив со следующими именами: `Sasha`, `Ivan`, `John`, `Orlando`, `Leonardo`, `Nina`, `Anton` и `Molly`. Попросите пользователя ввести имя. Используйте цикл foreach для проверки того, не находится ли имя, введенное пользователем, уже в массиве.

Пример результата выполнения программы:

```
Enter a name: Sasha
Sasha was found.

Enter a name: Masha
Masha was not found.
```

> ***Подсказка:*** Используйте `std::string` в качестве типа массива.

```
#include <iostream>
#include <string>

int main()
{
    const std::string names[] = { "Sasha", "Ivan", "John", "Orlando", "Leonardo", "Nina", "Anton", "Molly" };

    std::cout << "Enter a name: ";
    std::string username;
    std::cin >> username;

    bool found(false);
    for (const auto &name : names)
        if (name == username)
        {
            found = true;
            break;
        }

    if (found)
        std::cout << username << " was found.\n";
    else
        std::cout << username << " was not found.\n";

    return 0;
}
```